Object Oriented Programming

PRE-MID ASSIGNMENT

SUBMITTED BY :

FATEHYAB AJAZ

Student ID: F25BSCSOO4

SEMESTER - 2

SUBMITTED TO : MA'AM MAMONA

Date of submission : 27 February 2026

Marks Breakdown (15 total):

CLO	Assessment Criteria	Excellent (9-10)	Good (7-8)	Satisfactory (5-6)	Poor (0-4)	Points
CLO T1	**Understand OOP vs Procedural**	Clear, accurate answers showing understanding	Good understanding with minor errors	Basic understanding	Poor/no answers	5
CLO T2	**Implement classes with encapsulation**	Perfect encapsulation with proper private/public	Good encapsulation, minor issues	Basic encapsulation	Poor/ no encapsulation	10
CLO L1	**Use constructors & destructors**	Multiple constructors, proper destructor	Good implementation with 1-2 issues	Basic implementation	Missing/ incorrect	10
CLO L1	**Apply OOP relationships**	Perfect composition relationship	Good implementation	Basic composition	No proper composition	10
CLO L1	**Use OOP features**	Both features correctly implemented	One feature implemented well	Basic implementation	Missing features	5
CLO L1	**Develop working program**	Full menu, all features working	Most features working	Basic functionality	Doesn't work	5
Total						**45**

Question 1: Smart Room Manager.

1. Classes Used

- **Device class:** Represents a single electronic device.
    - **Attributes (private):** name, ison, brightness

```
// Class representing device
class Device {
private:
    string name;       // Device name
    bool ison;         // Device ON/OFF status
    int brightness;    // Brightness level (0-100)
```

    - **Methods (public):**
        - Device() – default constructor
        - Device(string, bool, int) – parameterized constructor
        - ~Device() – destructor
        - Getters/Setters
        - turnOn(), turnOff(), showInfo()
- **Room class:** Represents a room containing multiple devices.
    - **Attributes:** roomName, devices[5], count

```
// Class representing a room containing multiple devices
class Room {
private:
    string roomName;       // Name of the room
    Device devices[5];     // Array to store up to 5 devices
    int count;             // Current number of devices in the room
```

    - **Methods:**
        - addDevice() – adds a device to the room
        - showAllDevices() – prints all devices
        - findDevice() – finds a device by name
        - toggleDevice() – turns a device ON or OFF

- setDeviceBrightness() – changes brightness of a device

2. How my Code Works

- The **main function** shows a menu where the user can:
  1. Add a device
  2. Show all devices
  3. Toggle a device ON/OFF
  4. Change device brightness
  5. Exit the program
- User input is taken using cin, and the Room object manages all the devices.

```
do {
    // Menu
    cout << "=== Smart Room Manager ===" << endl;
    cout << "1. Add a Device" << endl;
    cout << "2.Show All Devices" << endl;
    cout << "3.Turn Device ON / OFF" << endl;
    cout << "4.Change Brightness" << endl;
    cout << "5.Exit" << endl;
    cout << "Enter choice" << endl;
    cin >> choice;
```

3. Constructors and Destructors

- **Constructor:** Used to **initialize objects**.
- **In my code:**

```
Device(string aname, bool aison, int abrightness) {
    name = aname;
    ison = aison;
    setBrightness(abrightness);  // Set brightness using setter to ensure it's valid
}
```

This sets the device's name, status, and brightness when the object is created.

- **Destructor:** Automatically runs when an object goes out of scope.

```
// Destructor
~Device() {
    cout << "Device " << name << " is being removed." << endl;
}
```

4. Composition and Aggregation

These are **object-oriented relationships** between classes.

- **Aggregation:** "Has-a" relationship where one class contains objects of another class, but the objects can exist independently.
  Example in my code:

```
Device d(name, status, brightness);
r.addDevice(d);
```

Here, the Room **aggregates** Device objects. Devices could exist outside a Room, so this is aggregation.

- **Composition:** "Strong has-a" relationship where the contained objects cannot exist independently of the container.
  Example in my code:

```
    Device devices[5];    // Array to store up to 5 devices
```

The Room **composes** an array of Devices. If the Room is destroyed, the array of Devices is destroyed too.

5. How Methods Work

- setBrightness(int) – validates the brightness (0–100)
- toggleDevice(string) – switches device ON/OFF
- showAllDevices() – loops through the array and prints info
- findDevice(string) – searches for a device by name

Question 2: Object-Oriented Design Analysis.

1. Identification of the the main classes with brief description.

Class	**Responsibility**
Product	Represents an item sold on the platform. Keeps track of name, ID, price, and stock quantity.
Customer	Represents a user who can browse products and place orders. Tracks personal info and purchase history.
Order	Represents a customer's order. Contains a list of products, total amount, and order status.
ShoppingCart	Represents a customer's current selections before placing an order. Can add/remove products and calculate totals.
Payment	Handles payment for an order. Tracks amount, payment method, and status.

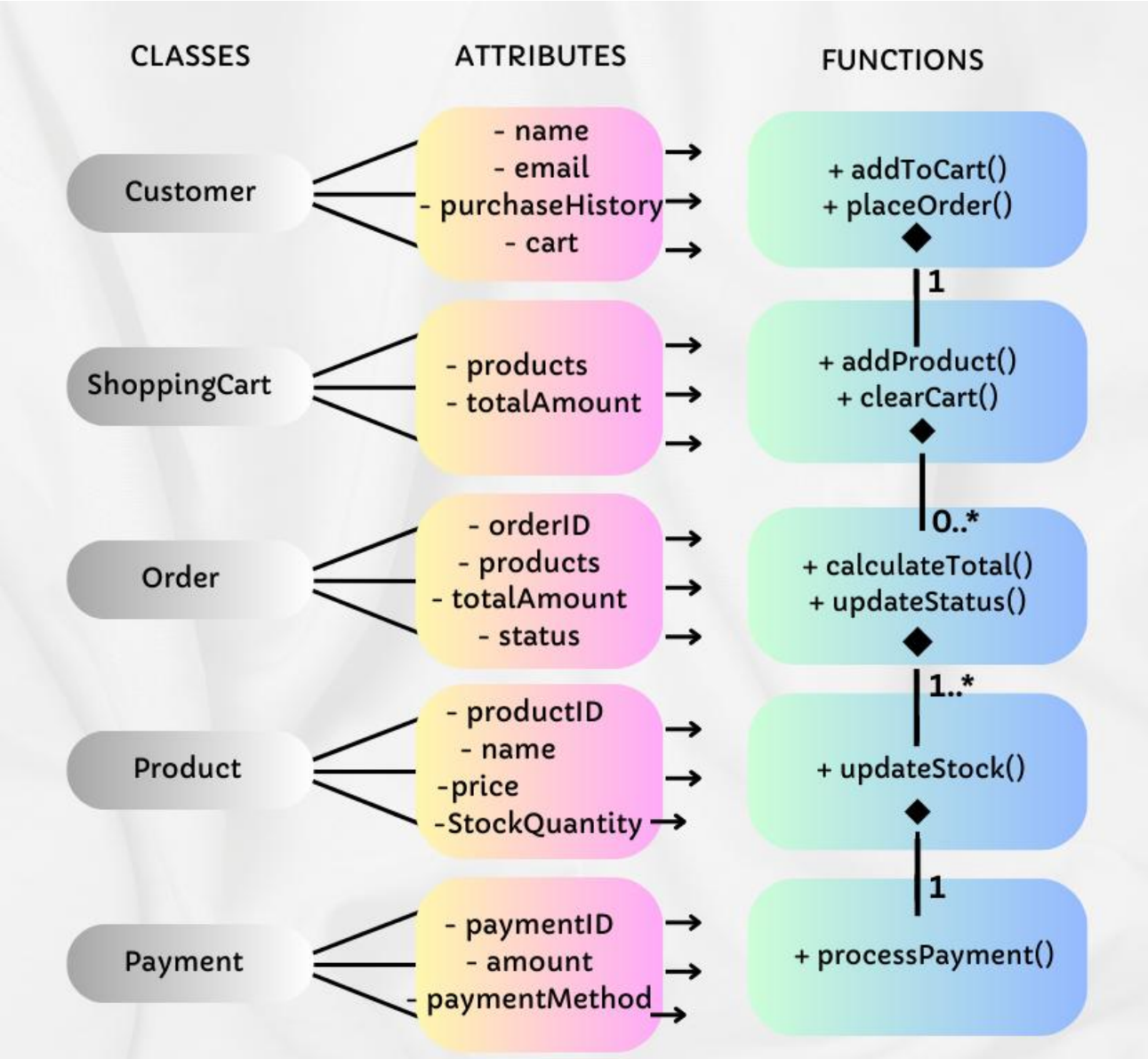
2. List for the key data members (attributes) for each class.

Class	**Attributes**
Product	**string** name, **int** id, **double** price, **int** stockQuantity
Customer	**string** name, **string** email**, string** purchaseHistory, **string** cart.
Order	**int** orderID, **string** products, **double** totalAmount, **string** status
ShoppingCart	**string** items, **double** totalAmount
Payment	**int** paymentID, **double** amount, **string** method, **string** status

3. List of the key member functions (methods) for each class.

Class	Key Member Functions (Methods)
Product	• updateStock(int quantity) • getPrice() • getProductDetails()
Customer	• addToCart(Product p) • removeFromCart(Product p) • placeOrder() • viewPurchaseHistory()
ShoppingCart	• addProduct(Product p) • removeProduct(Product p) • calculateTotal() • clearCart()
Order	• calculateTotal() • updateStatus(string newStatus) • getOrderDetails()
Payment	• processPayment() • validatePayment() • getPaymentStatus()

4. UML diagram.



Relationship	Type	Multiplicity	Reason
Customer → ShoppingCart	◆ Composition	1 → 1	Cart depends fully on Customer
Customer → Order	◇ Aggregation	1 → 0..*	Orders belong to Customer but can exist independently
Order → Product	◇ Aggregation	1 → 1..*	Products exist independently
Order → Payment	◆ Composition	1 → 1	Payment depends fully on Order

5.Skeleton structure of one class.

```cpp
#include <iosteam>
#include <string>
using namespace std;
class Product {
private:
    string name;
    int id;
    double price;
    int stockQuantity;
public:
    Product();
    Product(string name, int id, double price, int stockQuantity);
    ~Product();
    string getName() const;
    int getID() const;
    double getPrice() const;
    int getStockQuantity() const;
    void setName(string name);
    void setPrice(double price);
    void setStockQuantity(int quantity);
    void reduceStock(int quantity);
    void showInfo() const;
};
```